CS484 Project Final Report: Landmark Classification Using Resnet-18 Cifar Model

Mohammadhassan Mahmodi, Mehmet Paşa, Ömer Faruk Akbay
Electrical and Electronics Engineering Department, Bilkent University, 06800 Ankara, Turkey

***Abstract*—In this project, we classify land use satellite images, which play a critical role in fields such as environmental monitoring and urban analysis. By using the " EuroSAT Dataset" and optimizing model hyperparameters, we achieved significant accuracy in classifying ten different land types. The "ResNet-18 Cifar" model, adapted explicitly for small-scale images and featuring a 3x3 convolutional layer, was chosen as the model architecture. We reached a classification accuracy of 97.9% on the test dataset. The results demonstrate the effectiveness of customized residual networks in satellite image classification tasks and the importance of suitable hyperparameter optimization for achieving model success. Index Terms—computer vision, deep learning, ResNet-18, detection, classification, EuroSAT, satellite images.**

I. Introduction

Land-use classification from satellite imagery is a crucial task in remote sensing. The concept is applied in various fields, including environmental monitoring and urban analysis. Due to the visual similarity between classes and the low resolution of the images, this task necessitates models that can extract fine-grained spatial features.

In this project, we used the ResNet-18 Cifar model to classify the ten EuroSAT land-use categories. The model's residual connections and lightweight architecture make it suitable for small-scale image recognition. We train and evaluate the model under different hyperparameter settings to analyze its convergence behavior and overall classification performance. The results demonstrate the effectiveness of residual networks for satellite image recognition, highlighting the impact of training duration and learning rate on model accuracy.

II. Problem definition

In this project, we develop a deep learning algorithm that enables the identification of ten distinct land use classes in satellite images using the EuroSAT dataset. When performing image classification, several difficulties are encountered, including low image resolution, substantial intra-class variation, overlap of distinct classes, and atmospheric distortions. To perform image classification efficiently, we need an algorithm that enables the identification of distinct spatial image features while simultaneously resisting image noise and minute image details. To achieve this, we choose to fine-tune the ResNet-18 CIFAR algorithm, which is specifically employed in cases of small images.

III. Dataset

The EuroSAT dataset is a popular remote sensing benchmark. It consists of 27,000 RGB images (64 x 64 pixels) from Sentinel-2 satellite imagery. It comprises approximately 10 balanced land-use categories, including forest, residential, marine, and agricultural areas. Due to its diversity, EuroSAT is a key tool for evaluating and comparing deep learning models in satellite image recognition and classification [1].

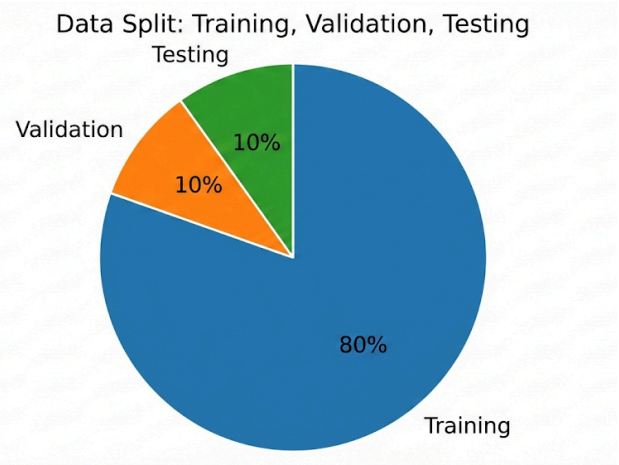

Fig. 1. Data Split Strategy

IV. Model architecture

ResNet-18, introduced initially by He et al., is a convolutional neural network that uses residual learning to make deeper models trainable and more accurate [2]. Instead of learning a direct mapping, each block learns a residual function that is added back to the input via a skip connection. In the project, a CIFAR-style variant of ResNet-18 is employed, specifically adapted for small-resolution images by changing the stem layer and

preserving higher spatial detail in the early stages of the network. The architecture can be expressed as a set of convolutional, normalization, activation, pooling, and fully connected operations organized in a residual hierarchy that enables efficient feature extraction, stable optimization, and strong generalization performance across diverse visual datasets.

The network begins with a simplified stem consisting of a single 3x3 convolutional layer, instead of a 7x7 convolution or max-pooling layer, as in the ImageNet-scale ResNet. This 3x3 convolution layer maps the input RGB image into 64 feature channels, followed by batch normalization and ReLU activation, which prevents overly reducing the spatial resolution of small images. Instead, the architecture of Cifar-style ResNet preserves fine-grained information early in the forward pass to support more effective learning on compact datasets. After the stem, the architecture transitions into a series of four primary residual stages that expand the feature dimensionality to 64, 128, 256, and 512 channels while progressively decreasing spatial resolution through strided convolutions.

Each stage is composed of two BasicBlocks, and each BasicBlock contains two consecutive 3x3 convolutional layers, both followed by batch normalization, with a ReLU applied after the first convolution. The main feature of the ResNet family is its skipping connections, which add the input of the block directly to its output. This procedure helps to train deeper networks without suffering from vanishing gradients. When the spatial resolution or number of channels changes between blocks (as is the case at the start of stages 2, 3, and 4), a 1x1 convolution with a stride of two is applied on the shortcut path to ensure compatibility with the main branch. This configuration allows the network to deepen its representational capacity while maintaining stable optimization behavior.

As the model progresses through its four stages, the first block in each stage employs a stride of 2 to downsample the feature maps, thereby increasing the spatial context available to the network to recognize higher-level and more abstract patterns. The remaining blocks in each stage use a stride of 1, preserving resolution and focusing on deepening the feature representation through additional nonlinear transformations. This gradual combination of downsampling and residual learning results in a balanced architecture that can capture both local texture patterns and global contextual information.

After passing through the final residual stage, the network applies an adaptive average pooling operation that reduces each feature map to a single scalar, resulting in a fixed-length feature vector of size 512, for all input dimensions. This vector is then passed through a dropout layer, which provides regularization by preventing overfitting, and finally through a fully connected linear layer that maps the learned representation to the ten classes defined in the EuroSAT dataset. These final-layer activations correspond to the model's estimates for each class. The complete architecture reflects a streamlined and computationally efficient residual network design, explicitly optimized for small images.

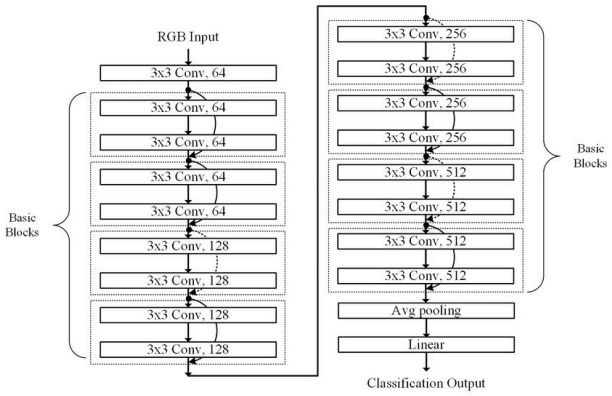

Fig. 2. Architecture of Resnet-18 Cifar Model

V. Hyperparameter tuning

Several important hyperparameters that significantly impact the performance of deep neural network training were selected for optimization during the hyperparameter tuning stage. The number of epochs, dropout, learning rate, batch size, weight decay, and width multiplier are some of these hyperparameters.

The network's convergence behavior and generalization capacity are directly impacted by the hyperparameters that are chosen. The total number of training iterations and the extent to which the model learns from the data are determined by the number of epochs. A key factor in striking a balance between training stability and convergence speed is the learning rate, which controls the size of parameter updates during optimization. Both convergence dynamics and computational efficiency are directly impacted by batch size. Also, it influences the stochasticity of gradient updates. By penalizing large model weights, weight decay functions as a regularization term to help avoid overfitting. Lastly, the width multiplier allows a trade-off between model complexity and performance by modifying the network capacity by scaling the number of channels.

Classification accuracy was chosen as the optimization goal since it accurately represents the model's overall performance on the CIFAR dataset. The definition of accuracy is

$$\text{Accuracy} = (TP + TN) / (FP + FN + TP + TN)$$

where TP, TN, FP, and FN stand for true positives, true negatives, false positives, and false negatives, in that order. To further distinguish between adequate and suboptimal hyperparameter configurations, a performance threshold of $\tau = 85\%$ was implemented during the optimization process.

The selection of the hyperparameter search strategy was discussed after the optimization objectives were established. Grid search is a simple method that thoroughly assesses every possible combination of hyperparameters. This ensures that the ideal configuration will be found, but given the available computational resources, it is impractical and computationally costly. Although random search is a more computationally efficient alternative, it does not consistently converge to optimal solutions. Instead, it samples random points from the hyperparameter space.

Bayesian Optimization was chosen as a fair and effective search method to get around these restrictions. Bayesian Optimization enables an efficient trade-off between exploration and exploitation of the hyperparameter space by iteratively updating a probabilistic model of the objective function based on prior evaluations, as illustrated in Figure 2.

Fig. 3. Graphical comparison of search algorithms

The Optuna framework was used to carry out the Bayesian optimization procedure. Optuna automatically controls hyperparameter configuration sampling and evaluation, and it modifies its search approach in response to performance data. The optimization process in this study was conducted over 30 trials, which corresponded to training 30 distinct ResNet-18 model instances on the CIFAR dataset.

The optimization results demonstrate the effectiveness of the chosen strategy. When the predetermined threshold was applied, the accuracy of the optimal configuration was 87.9%. However, when the threshold constraint was not used, the accuracy of the same configuration increased to 97.9%. These findings demonstrate that, in a limited number of trials, the Bayesian optimization framework successfully identified high-performing hyperparameter configurations.

VI. Training and Results

The EuroSAT dataset was utilized for all experiments. To guarantee a balanced evaluation and reliable generalization, the dataset was systematically partitioned into distinct sets: 80% for training, 10% for validation, and 10% for testing.

Training run #1 produces a successful outcome, achieving a final accuracy of 96.19%. The configuration was as follows:

- Epochs: 10
- Batch size: 64
- Learning rate: 1×10^{-3}
- Weight decay: 1×10^{-4}
- Dropout: 0.2
- Width multiplier: 1.0

(These values are our ground truth for the following experiments)

Effect of Epoch Number

Training #2 was executed with the identical hyperparameters as Training #1, but was limited to 5 epochs to assess its effect on the training. This configuration produces a final accuracy of 92.41%, which was achieved at epoch 5. Although the model's performance remains strong, it did not match the accuracy of the first training, which illustrates that this learning rate (1×10^{-3}) requires a longer training time for the model to fully converge.

Effect of Learning Rate (LR)

In the third run, the model was trained for 10 epochs with a learning rate increased to 1×10^{-1}, while all other parameters remained unchanged. This higher learning rate resulted in unstable weight updates and fluctuating loss values, which resulted in a lower accuracy of 91.41%. We observed that excessively large learning rates can prevent convergence and negatively affect model generalization.

Remark: After this, to better isolate the influence of individual hyperparameters—which was obscured by the model's high overall accuracy—we implemented a confidence threshold. This approach limited the performance evaluation to only those predictions that achieved a confidence score above 85% (0.85).

With the values of training #1, the achieved accuracy was 85%.

Effect of Width Multiplier

In our fourth run, we decreased the width multiplier from 1.0 to 0.15. As expected, the accuracy dropped to 80% from 85%, since high width multipliers significantly lowers the model's representational capacity. With fewer neurons, the model cannot capture complex spatial and semantic features effectively, which leads to underfitting.

Effect of Weight Decay

In our fifth run, we set the weight decay to zero. As a result, the accuracy dropped to 81% from 85%, since removing weight decay eliminates an important form of regularization. Without this restriction, the model becomes more prone to overfitting, learning noise and dataset-specific patterns rather than generalizable features, which results in reduced performance on unseen data.

Effect of Batch size

In training #6, the batch size was increased to 256. As the output, the accuracy dropped to 79.7% from 85%, since larger batch sizes reduce gradient noise and can lead to poorer generalization. With less stochasticity during optimization, the model may converge to sharper minima, which negatively affects performance on test data.

Effect of dropout

In training #7, the dropout rate was increased to 0.7. As expected, the accuracy dropped to 78.6% from 85%, since high dropout removes too many active neurons during training. This strong regularization restricts the model's ability to learn robust feature representations, which leads to underfitting and reduced accuracy.

Best Result after Fine-tuning

The optimal hyperparameters obtained after 30 Optuna trials are as follows:

- Epochs: 10 - Batch size: 32 - Learning rate: 5.2×10^{-3} - Weight decay: 1.024×10^{-6}
- Dropout: 0.14 - Width multiplier: 0.75

After hyperparameter tuning using Optuna with 30 trials, the model achieved its best performance with an accuracy of 87.9% when applying a confidence threshold, and 97.9% accuracy without applying the threshold, which is our most successful result.

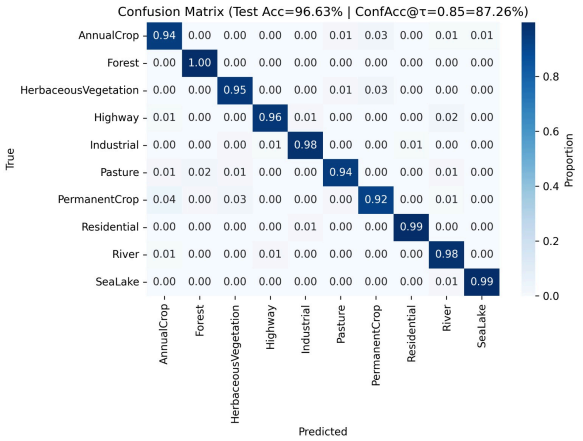

Fig. 4. Confusion matrix of the results on the test set

Fig. 5. Performance of Resnet-18 Cifar on various metrics

The training demonstrated stable convergence, as evidenced by the smooth loss and accuracy curves, which indicates appropriate tuning of the learning rate and weight decay for the dataset. Furthermore, key validation metrics, including Precision, Recall, and the F1 score, stabilized after the sixth epoch, indicating that a total of ten epochs was adequate for achieving full convergence.

Fig. 6. Land classification performance of the best training

VII. Baseline comparison

A comprehensive research is conducted to find other models trained on the same dataset. As a result, a table of models is presented, as shown in Figure 6. The table includes both Transformer-Based Models and

Convolutional Neural Networks-based models [3]. Our model's accuracy without hyperparameter tuning is 96.19%, which is average according to pre-trained accuracy results. Our hyperparameter-tuned model achieved an accuracy of 97.9%, surpassing most state-of-the-art models while remaining relatively lightweight, particularly in comparison to the Transformer-based model.

Model name	Accuracy	Accuracy pre-trained
AlexNet	0.837	0.916
ResNet	0.835	0.947
ResNeXt	0.843	0.981
DenseNet	0.925	0.949
MobileNetV3	0.877	0.958
EfficientNetV2	0.908	0.973
ConvNeXt	0.783	0.986
ViT32	0.917	0.972
SwinB	0.926	0.987
MaxViT	**0.973**	**0.990**

Fig. 7. Classification accuracy comparison on ten deep learning models

VIII. Conclusion

In conclusion, this project demonstrates that by applying appropriate hyperparameters, the accuracy of the ResNet-18 Cifar model can be considerably increased. We utilized Bayesian optimization with the Optuna library to enhance hyperparameter selection performance in 30 experiments, resulting in significant time savings and a more reliable model.

Our pre-trained model achieves 96.19% accuracy without requiring hyperparameter tuning. To analyze the effects of hyperparameters more accurately, we used a performance threshold of 85%. After adjusting the hyperparameters, our model achieved an accuracy of 97.9% and surpassed most state-of-the-art models. As a future goal, this project can be extended to more complex tasks, such as semantic segmentation or object detection, to locate specific structures or boundaries within land-use categories. Ultimately, our project demonstrates that using suitable hyperparameters is crucial for enhancing the accuracy of land-use classification models.

IX. About the code

In the code.zip, training and test can be found. The eurosat dataset can be accessed via the github link provided in references. The weight file is also accessible, which is denoted as “best.pt”.

REFERENCES

[1] P. Helber, B. Bischke, A. Dengel, and D. Borth, “EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification,” IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens., vol. 12, no. 7, pp. 2217–2226, 2019.

[2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 770–778, 2016.

[3] J. Terven, "eurosat_classification," *GitHub*. [Online]. Available: https://github.com/jrterven/eurosat_classification/tree/main. [Accessed: Dec. 24, 2025].